Colab (d) — PyTorch Lightning 3-Layer Deep Neural Network (Nonlinear Regression)

Goal: Implement the same **3-layer** nonlinear regression model using **PyTorch Lightning**.

✅ Requirements covered:

- Synthetic data from a **3-variable nonlinear equation** `(x1, x2, x3) -> y`
- **3-layer** network: `3 → H1 → H2 → 1`
- Proper **nonlinear activations**
- Uses **PyTorch Lightning** (`LightningModule`, `Trainer`)
- Training curve (loss vs epochs) via logger + final evaluation
- "4D plot": PCA to 3D + color = y

0. Setup

```
# If needed in Colab:
!pip -q install torch torchvision torchaudio
!pip -q install pytorch-lightning torchmetrics
!pip -q install numpy matplotlib scikit-learn pandas

import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

import pytorch_lightning as pl
from pytorch_lightning.loggers import CSVLogger

import matplotlib.pyplot as plt
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

np.random.seed(42)
torch.manual_seed(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Torch:", torch.__version__)
print("PyTorch Lightning:", pl.__version__)
print("CUDA available:", torch.cuda.is_available())
```

```
Torch: 2.10.0+cu128
PyTorch Lightning: 2.6.1
CUDA available: True
```

1. Generate synthetic nonlinear regression data (3 variables)

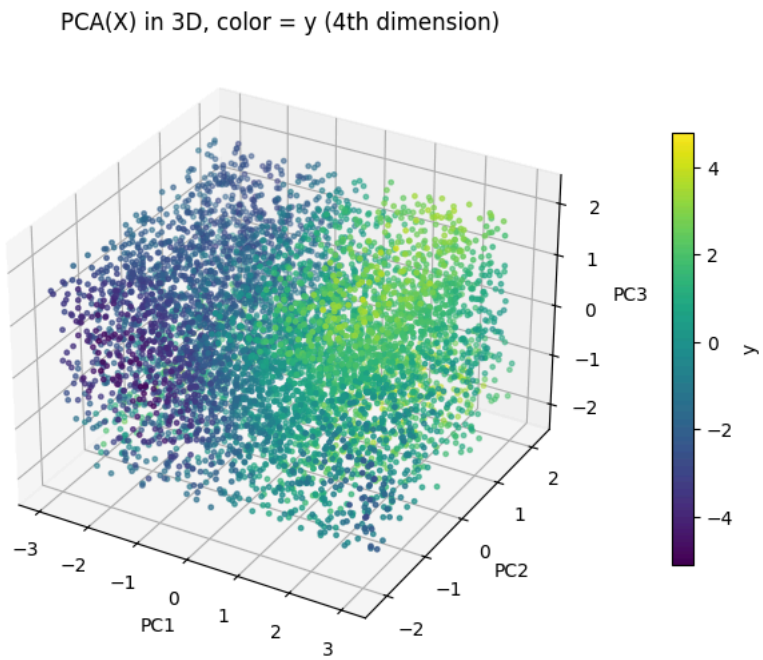
```
def nonlinear_equation(x1, x2, x3):
    return (
        2.0 * np.sin(x1)
        + 0.5 * (x2 ** 2)
        - 3.0 * np.exp(-x3)
        + 0.3 * x1 * x2
        - 0.2 * x2 * x3
    )

N = 6000
x1 = np.random.uniform(-3.0, 3.0, size=(N, 1))
x2 = np.random.uniform(-2.0, 2.0, size=(N, 1))
x3 = np.random.uniform(0.0, 4.0, size=(N, 1))

X = np.concatenate([x1, x2, x3], axis=1)
noise = np.random.normal(0.0, 0.2, size=(N, 1))
y = nonlinear_equation(x1, x2, x3).reshape(-1, 1) + noise
```

```
print("X shape:", X.shape, "y shape:", y.shape)
```

```
X shape: (6000, 3) y shape: (6000, 1)
```

2. Quick visualization ("4D" plot via PCA + color = y)

```
pca = PCA(n_components=3, random_state=42)
X_pca = pca.fit_transform(X)

fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
sc = ax.scatter(X_pca[:, 0], X_pca[:, 1], X_pca[:, 2], c=y[:, 0], s=6)
ax.set_title("PCA(X) in 3D, color = y (4th dimension)")
ax.set_xlabel("PC1"); ax.set_ylabel("PC2"); ax.set_zlabel("PC3")
plt.colorbar(sc, ax=ax, pad=0.1, shrink=0.7, label="y")
plt.show()
```



3. Train/Val/Test split + standardization

```
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.50, random_state=42)

x_scaler = StandardScaler()
X_train_s = x_scaler.fit_transform(X_train)
X_val_s   = x_scaler.transform(X_val)
X_test_s  = x_scaler.transform(X_test)

y_scaler = StandardScaler()
y_train_s = y_scaler.fit_transform(y_train)
y_val_s   = y_scaler.transform(y_val)
y_test_s  = y_scaler.transform(y_test)

print("Train:", X_train_s.shape, y_train_s.shape)
print("Val  :", X_val_s.shape, y_val_s.shape)
print("Test :", X_test_s.shape, y_test_s.shape)
```

```
Train: (4200, 3) (4200, 1)
Val  : (900, 3) (900, 1)
Test : (900, 3) (900, 1)
```

4. Dataset + DataLoaders

```
class RegressionDataset(Dataset):
    def __init__(self, X_np, y_np):
        self.X = torch.tensor(X_np, dtype=torch.float32)
        self.y = torch.tensor(y_np, dtype=torch.float32)

    def __len__(self):
        return self.X.shape[0]

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]

batch_size = 128
train_loader = DataLoader(RegressionDataset(X_train_s, y_train_s), batch_size=batch_size, shuffle=True, num_workers=
val_loader   = DataLoader(RegressionDataset(X_val_s, y_val_s),     batch_size=batch_size, shuffle=False, num_workers
test_loader  = DataLoader(RegressionDataset(X_test_s, y_test_s),   batch_size=batch_size, shuffle=False, num_workers
```

5. LightningModule (3-layer model + training/validation/test steps)

```
class LightningThreeLayerRegressor(pl.LightningModule):
    def __init__(self, d_in=3, h1=64, h2=32, d_out=1, lr=1e-3):
        super().__init__()
        self.save_hyperparameters()

        self.net = nn.Sequential(
            nn.Linear(d_in, h1),
            nn.Tanh(),
            nn.Linear(h1, h2),
            nn.Tanh(),
            nn.Linear(h2, d_out),
        )

        self.criterion = nn.MSELoss()

    def forward(self, x):
        return self.net(x)

    def training_step(self, batch, batch_idx):
        x, y = batch
        yhat = self(x)
        loss = self.criterion(yhat, y)
        self.log("train_loss", loss, on_step=False, on_epoch=True, prog_bar=True)
        return loss

    def validation_step(self, batch, batch_idx):
        x, y = batch
        yhat = self(x)
        loss = self.criterion(yhat, y)
        self.log("val_loss", loss, on_step=False, on_epoch=True, prog_bar=True)
        return loss

    def test_step(self, batch, batch_idx):
        x, y = batch
        yhat = self(x)
        loss = self.criterion(yhat, y)
        self.log("test_loss", loss, on_step=False, on_epoch=True, prog_bar=True)
        return loss

    def configure_optimizers(self):
        return torch.optim.Adam(self.parameters(), lr=self.hparams.lr)

model = LightningThreeLayerRegressor(d_in=3, h1=64, h2=32, d_out=1, lr=1e-3)
print(model)
```

```
LightningThreeLayerRegressor(
  (net): Sequential(
    (0): Linear(in_features=3, out_features=64, bias=True)
    (1): Tanh()
    (2): Linear(in_features=64, out_features=32, bias=True)
    (3): Tanh()
```

```
    (4): Linear(in_features=32, out_features=1, bias=True)
  )
  (criterion): MSELoss()
)
```

6. Train with Lightning Trainer + CSV logger

```
pl.seed_everything(42, workers=True)

logger = CSVLogger(save_dir="lightning_logs", name="regression_3layer")

trainer = pl.Trainer(
    max_epochs=250,
    accelerator="gpu" if torch.cuda.is_available() else "cpu",
    devices=1,
    logger=logger,
    log_every_n_steps=10,
    enable_checkpointing=False,
)

trainer.fit(model, train_loader, val_loader)
```

```
INFO:lightning_fabric.utilities.seed:Seed set to 42
INFO:pytorch_lightning.utilities.rank_zero:GPU available: True (cuda), used: True
INFO:pytorch_lightning.utilities.rank_zero:TPU available: False, using: 0 TPU cores
INFO:pytorch_lightning.utilities.rank_zero:💡 Tip: For seamless cloud logging and experiment tracking, try installing
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
```

	Name	Type	Params	Mode	FLOPs
0	net	Sequential	2.4 K	train	0
1	criterion	MSELoss	0	train	0

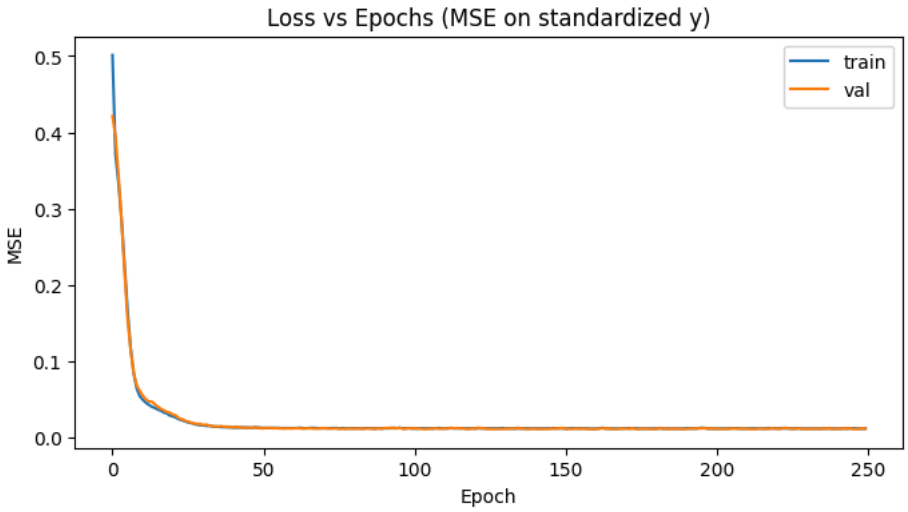
```
Trainable params: 2.4 K
Non-trainable params: 0
Total params: 2.4 K
Total estimated model params size (MB): 0
Modules in train mode: 7
Modules in eval mode: 0
Total FLOPs: 0
Epoch 249/249 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 33/33 0:00:00 • 0:00:00 108.57it/s v_num: 1.000 val_loss: 0.011
                                                                                          train_loss: 0.012
/usr/local/lib/python3.12/dist-packages/pytorch_lightning/utilities/_pytree.py:21: `isinstance(treespec, LeafSpec)`
is deprecated, use `isinstance(treespec, TreeSpec) and treespec.is_leaf()` instead.
/usr/local/lib/python3.12/dist-packages/pytorch_lightning/trainer/connectors/data_connector.py:434: The
'val_dataloader' does not have many workers which may be a bottleneck. Consider increasing the value of the
`num_workers` argument` to `num_workers=11` in the `DataLoader` to improve performance.
/usr/local/lib/python3.12/dist-packages/pytorch_lightning/trainer/connectors/data_connector.py:434: The
'train_dataloader' does not have many workers which may be a bottleneck. Consider increasing the value of the
`num_workers` argument` to `num_workers=11` in the `DataLoader` to improve performance.
INFO:pytorch_lightning.utilities.rank_zero:`Trainer.fit` stopped: `max_epochs=250` reached.
```

7. Plot train/val loss curves from the CSV logs

```
# Lightning writes a metrics.csv file. We'll read it and plot epoch-level metrics.
metrics_path = f"{logger.log_dir}/metrics.csv"
print("Reading:", metrics_path)

df = pd.read_csv(metrics_path)

# Keep only rows where epoch metrics are present
# (train_loss_epoch / val_loss are logged on epoch)
df_epoch = df.dropna(subset=["epoch"]).copy()

# Get last logged value per epoch
train_by_epoch = df_epoch.dropna(subset=["train_loss"]).groupby("epoch")["train_loss"].last()
val_by_epoch   = df_epoch.dropna(subset=["val_loss"]).groupby("epoch")["val_loss"].last()

plt.figure(figsize=(8, 4))
plt.plot(train_by_epoch.index, train_by_epoch.values, label="train")
plt.plot(val_by_epoch.index, val_by_epoch.values, label="val")
plt.title("Loss vs Epochs (MSE on standardized y)")
plt.xlabel("Epoch")
plt.ylabel("MSE")
```

```
plt.legend()
plt.show()

print("Final epoch train_loss:", float(train_by_epoch.values[-1]))
print("Final epoch val_loss  :", float(val_by_epoch.values[-1]))
```

```
Reading: lightning_logs/regression_3layer/version_1/metrics.csv
```




```
Final epoch train_loss: 0.0117993662133812
Final epoch val_loss  : 0.0114193353801965
```

8. Test set evaluation + true vs predicted (original y scale)

```
# Run test loop (reports standardized-space test_loss)
test_results = trainer.test(model, dataloaders=test_loader, verbose=False)
print("Lightning test results:", test_results)

# Also compute predictions, inverse-transform to original y scale, and plot.
model.eval()
model.to(device)

X_test_t = torch.tensor(X_test_s, dtype=torch.float32, device=device)
with torch.no_grad():
    y_test_hat_s = model(X_test_t).detach().cpu().numpy()

y_test_hat = y_scaler.inverse_transform(y_test_hat_s)
y_test_true = y_test  # already original scale

test_mse = float(np.mean((y_test_hat - y_test_true) ** 2))
print("Final TEST MSE (original y scale):", test_mse)

plt.figure(figsize=(6, 6))
plt.scatter(y_test_true[:, 0], y_test_hat[:, 0], s=10)
plt.title("True vs Predicted (Test)")
plt.xlabel("y_true")
plt.ylabel("y_pred")
minv = min(y_test_true.min(), y_test_hat.min())
maxv = max(y_test_true.max(), y_test_hat.max())
plt.plot([minv, maxv], [minv, maxv])
plt.show()

for i in range(10):
    print(f"Example {i}: y_true={y_test_true[i,0]: .3f} | y_pred={y_test_hat[i,0]: .3f}")
```

```
INFO:pytorch_lightning.accelerators.cuda:LOCAL_RANK: 0 - CUDA_VISIBLE_DEVICES: [0]
Testing ━━━
```